

# AI SERVICE DESK

## Technical Documentation

Date: July 24, 2026 | Version: 1.0

## Revision History

| Version | Date        | Author    | Description                            |
|---------|-------------|-----------|----------------------------------------|
| 1.0     | 24-Jul-2026 | Abinaya K | Documentation on Python training tasks |

## Table of Contents

|                                       |    |
|---------------------------------------|----|
| Revision History .....                | 2  |
| Table of Contents .....               | 2  |
| 1. Introduction .....                 | 5  |
| 1.1 Purpose .....                     | 5  |
| 1.2 Scope .....                       | 5  |
| 1.3 Intended Audience.....            | 5  |
| 1.4 References .....                  | 5  |
| 1.5 Acronyms and Definitions.....     | 5  |
| 2. System Overview.....               | 6  |
| 2.1 High-Level Architecture.....      | 6  |
| 2.2 System flow - Key Components..... | 6  |
| Ticket CRUD Request Flow.....         | 7  |
| AI Summarization Flow .....           | 7  |
| 2.3 User Roles and Access Levels..... | 7  |
| 3. Functional Requirements.....       | 7  |
| 3.1 Module/Feature Name .....         | 7  |
| Ticket Management.....                | 7  |
| AI Ticket Summarization .....         | 7  |
| 3.2 Inputs/Outputs .....              | 8  |
| 3.3 Business Rules.....               | 8  |
| 3.4 Error Handling.....               | 8  |
| 4. Non-Functional Requirements.....   | 9  |
| 4.1 Performance.....                  | 9  |
| 4.2 Security.....                     | 9  |
| 4.3 Availability .....                | 9  |
| 4.4 Maintainability .....             | 9  |
| 5. Data Model / Database Design ..... | 9  |
| 5.1 Entity-Relationship Diagram.....  | 9  |
| 5.2 Tables and Fields.....            | 10 |
| 5.3 Indexes and Keys.....             | 10 |
| 5.4 Sample Data.....                  | 10 |
| 6. Integration Points.....            | 10 |
| 6.1 External Systems .....            | 10 |
| 6.2 APIs Used.....                    | 11 |
| 6.3 Data Flow Diagram .....           | 11 |

|                                              |                                     |
|----------------------------------------------|-------------------------------------|
| 6.4 Authentication/Authorization .....       | 11                                  |
| 7. Test Strategy.....                        | 12                                  |
| 7.1 Test Scenarios.....                      | 12                                  |
| 7.2 Expected Results .....                   | <b>Error! Bookmark not defined.</b> |
| 7.3 Traceability to Requirements .....       | 15                                  |
| 8. Deployment & Configuration.....           | 16                                  |
| 8.1 Environments.....                        | 16                                  |
| 8.2 Setup Steps .....                        | 16                                  |
| 8.3 Configuration Files and Parameters ..... | 17                                  |
| 9. Appendix .....                            | 17                                  |
| Glossary.....                                | 17                                  |
| Screenshots.....                             | 18                                  |
| Code Snippets.....                           | 20                                  |
| References .....                             | 23                                  |

## 1. Introduction

### 1.1 Purpose

This document describes the technical design and implementation of the AI Service Desk, a backend service that manages support tickets and augments them with AI-generated summaries and suggested responses. It is intended to give engineers, reviewers, and operators a single reference for how the system is built, how it behaves, and how to run and test it.

### 1.2 Scope

The scope of this document covers the FastAPI-based REST API for ticket management (create, read, update, delete), the AI summarization endpoint backed by Amazon Bedrock, the data model, database migrations, testing strategy, and deployment via Docker. It does not cover a front-end/UI (none exists in the repository) or any client applications that may consume this API.

### 1.3 Intended Audience

- Backend developers extending or maintaining the service
- QA engineers writing or reviewing automated tests
- DevOps engineers responsible for deployment and configuration
- Technical reviewers / project stakeholders evaluating the implementation

### 1.4 References

- Source repository: ai-service-task (FastAPI + SQLAlchemy + Amazon Bedrock)
- FastAPI documentation: <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 (async) documentation: <https://docs.sqlalchemy.org>
- Amazon Bedrock Converse API: <https://docs.aws.amazon.com/bedrock/>
- Alembic migrations: <https://alembic.sqlalchemy.org>

### 1.5 Acronyms and Definitions

| Term | Definition                        |
|------|-----------------------------------|
| API  | Application Programming Interface |
| REST | Representational State Transfer   |
| ORM  | Object-Relational Mapping         |
| CRUD | Create, Read, Update, Delete      |
| UUID | Universally Unique Identifier     |
| LLM  | Large Language Model              |

| Term | Definition                            |
|------|---------------------------------------|
| JSON | JavaScript Object Notation            |
| ASGI | Asynchronous Server Gateway Interface |
| CI   | Continuous Integration                |

## 2. System Overview

### 2.1 High-Level Architecture

The AI Service Desk is a single FastAPI application organized in layers: API routers, a service layer for business rules, a repository layer for persistence, and SQLAlchemy ORM models. Persistence uses an async SQLAlchemy engine (aiosqlite for local/testing, asyncpg or psycopg for PostgreSQL in other environments) with Alembic managing schema migrations. A separate AWS integration layer wraps the Amazon Bedrock Converse API to generate ticket summaries and suggested responses. The application is containerized with Docker and exposes health and readiness endpoints for orchestration.

Key building blocks:

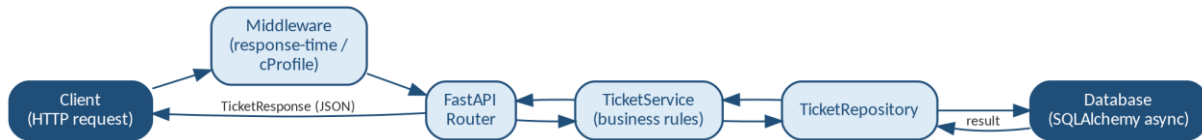
- FastAPI application (main.py) – wires routers, middleware, and startup/shutdown lifecycle
- app/api – HTTP route handlers for tickets and AI summarization
- app/services – business logic (TicketService) and the AWS Bedrock client wrapper (BedrockService)
- app/repositories – direct database access (TicketRepository)
- app/models / app/schemas – SQLAlchemy ORM models and Pydantic request/response schemas
- app/db – async engine/session setup and declarative base
- alembic – database migration scripts

### 2.2 System flow - Key Components

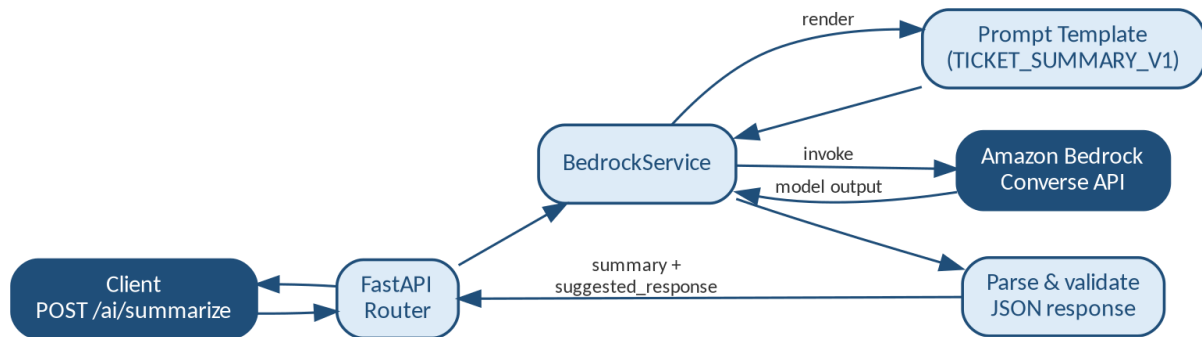
A typical request flows as follows: the client calls a REST endpoint → request passes through the response-time middleware (and optional cProfile middleware) → the FastAPI router → TicketService applies business rules → TicketRepository executes the SQLAlchemy query against the database → result is serialized via a Pydantic response schema back to the client.

For AI summarization, the /ai/summarize endpoint calls BedrockService, which renders a prompt template, invokes the Amazon Bedrock Converse API, parses the JSON response, and returns a structured summary and suggested reply. If Bedrock is unavailable or returns malformed output, a BedrockServiceError is raised and mapped to an HTTP 502 response.

## Ticket CRUD Request Flow



## AI Summarization Flow



## 2.3 User Roles and Access Levels

The current codebase does not implement authentication or role-based access control — all endpoints are publicly callable by any client that can reach the service. Configuration includes a `SECRET_KEY` and `ACCESS_TOKEN_EXPIRE_MINUTES` setting, suggesting token-based auth is planned, but no auth middleware or dependency is wired into the routers yet. This should be treated as a functional gap to address before any production/multi-tenant deployment.

# 3. Functional Requirements

## 3.1 Module/Feature Name

### *Ticket Management*

Provides full CRUD operations for support tickets, allowing tickets to be created, listed (with optional filtering), retrieved individually, updated, and deleted.

### *AI Ticket Summarization*

Accepts a free-text ticket description and returns an AI-generated summary and a suggested professional response, powered by an LLM served through Amazon Bedrock.

### 3.2 Inputs/Outputs

| Endpoint             | Method | Input                                           | Output                                                                |
|----------------------|--------|-------------------------------------------------|-----------------------------------------------------------------------|
| /tickets             | POST   | title (3-200 chars), priority (low medium high) | TicketResponse (id, title, priority, status, created_at, is_resolved) |
| /tickets             | GET    | Optional query params: status, priority         | List of TicketResponse                                                |
| /tickets/{ticket_id} | GET    | ticket_id (UUID, path)                          | TicketResponse or 404                                                 |
| /tickets/{ticket_id} | PATCH  | Any of: title, priority, status, assignee       | Updated TicketResponse                                                |
| /tickets/{ticket_id} | DELETE | ticket_id (UUID, path)                          | Confirmation message                                                  |
| /ai/summarize        | POST   | ticket_description (10-5000 chars)              | summary, suggested_response                                           |
| /health              | GET    | None                                            | {"status": "ok" / "healthy"}                                          |
| /ready               | GET    | None                                            | {"status": "ready"} or 503                                            |

### 3.3 Business Rules

- Ticket titles must be 3-200 characters and cannot be blank or whitespace-only (validated and trimmed on input).
- Priority must be one of: low, medium, high.
- Ticket status must be one of: open, in\_progress, resolved, closed (default: open).
- A closed ticket cannot be reopened — attempting to change its status away from “closed” raises a validation error (HTTP 400).
- Partial updates are supported: only the fields supplied in a PATCH request are modified (exclude\_unset).

### 3.4 Error Handling

- Requesting a ticket that does not exist raises TicketNotFoundError, handled globally and returned as HTTP 404 with { "error": "ticket\_not\_found", "id": <ticket\_id> }.
- Invalid business-rule transitions (e.g., reopening a closed ticket) raise ValueError, mapped to HTTP 400 with the error detail.
- Request payload validation failures (schema/type/length) are handled automatically by FastAPI/Pydantic and return HTTP 422.
- Bedrock/AI failures (network errors, malformed model output) raise BedrockServiceError and are returned as HTTP 502 with a generic “temporarily unavailable” message, without leaking internal details.
- Repository-level database errors trigger an explicit session rollback before the exception is re-raised.



## 4. Non-Functional Requirements

### 4.1 Performance

Every response includes an X-Response-Time header (measured in the outer middleware) for lightweight latency observability. An optional CProfileMiddleware can be enabled per-request (query param profile=true, header X-Profile: true, or env var ENABLE\_CPROFILE=true) to capture detailed cProfile statistics to the cprofile\_results/ directory. A dedicated cprofile\_runner.py script and a Locust load-testing file (locustfile.py) are included to profile and load-test the create, read, update, delete, and health endpoints.

### 4.2 Security

AWS credentials (AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY, AWS\_REGION) are supplied via environment variables / a .env file and used only server-side by the Bedrock client — they are never exposed to API consumers. Bedrock failures are caught and returned as a generic message rather than surfacing internal

exception details. As noted in section 2.3, the API itself currently has no authentication/authorization layer; the SECRET\_KEY and ACCESS\_TOKEN\_EXPIRE\_MINUTES settings are reserved for future token-based auth. It is recommended to add an authentication dependency before exposing this service outside a trusted network.

### 4.3 Availability

The service exposes /health (liveness) and /ready (readiness, which checks database connectivity) endpoints suitable for container orchestration probes. The Dockerfile defines a HEALTHCHECK that polls /health every 30 seconds. Note: a /health route is defined twice — once at the application root in main.py and once again inside the tickets router — which is functionally redundant and worth consolidating.

### 4.4 Maintainability

- Clear separation of concerns: API → Service → Repository → ORM Model layers.
- Schema evolution is managed through Alembic migrations rather than ad-hoc DDL.
- Dependency management via uv (pyproject.toml / uv.lock), with a generated requirements.txt for environments that need it.
- Automated unit tests (mocked DB) and integration tests (in-memory SQLite) cover create, read, update, and delete flows.
- The FakeBedrockService offline implementation allows the AI feature to be exercised deterministically without live AWS calls or cost.

## 5. Data Model / Database Design

### 5.1 Entity-Relationship Diagram

The current schema consists of a single entity, Ticket, with no foreign-key relationships to other tables. As the domain grows (e.g., adding Users, Comments, or Attachments), an ER diagram should be introduced to show those relationships; at present a single-entity description is sufficient.

## 5.2 Tables and Fields

| Field          | Type                | Constraints                                          | Description                                           |
|----------------|---------------------|------------------------------------------------------|-------------------------------------------------------|
| id             | UUID                | Primary key, default uuid4()                         | Unique ticket identifier                              |
| title          | String(200)         | NOT NULL                                             | Short ticket title/summary (3-200 chars at API level) |
| priority       | Enum                | low   medium   high, default medium                  | Ticket priority                                       |
| status         | Enum                | open   in_progress   resolved   closed, default open | Ticket lifecycle state                                |
| assignee       | String              | Nullable                                             | Name of the assigned agent                            |
| assignee_email | String(254)         | Nullable                                             | Email of the assigned agent                           |
| created_at     | DateTime (tz-aware) | Server default now()                                 | Ticket creation timestamp                             |

## 5.3 Indexes and Keys

The id column is the table's primary key (UUID). No additional indexes are currently defined on priority or status, even though both are used as filter parameters on GET /tickets. As ticket volume grows, adding indexes on these columns — and on assignee if agent-based lookups become common — is recommended to keep filtered list queries performant.

## 5.4 Sample Data

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "title": "Unable to log in after password reset",
  "priority": "high",
  "status": "open",
  "created_at": "2026-07-23T09:15:00Z",
  "is_resolved": false
}
```

# 6. Integration Points

## 6.1 External Systems

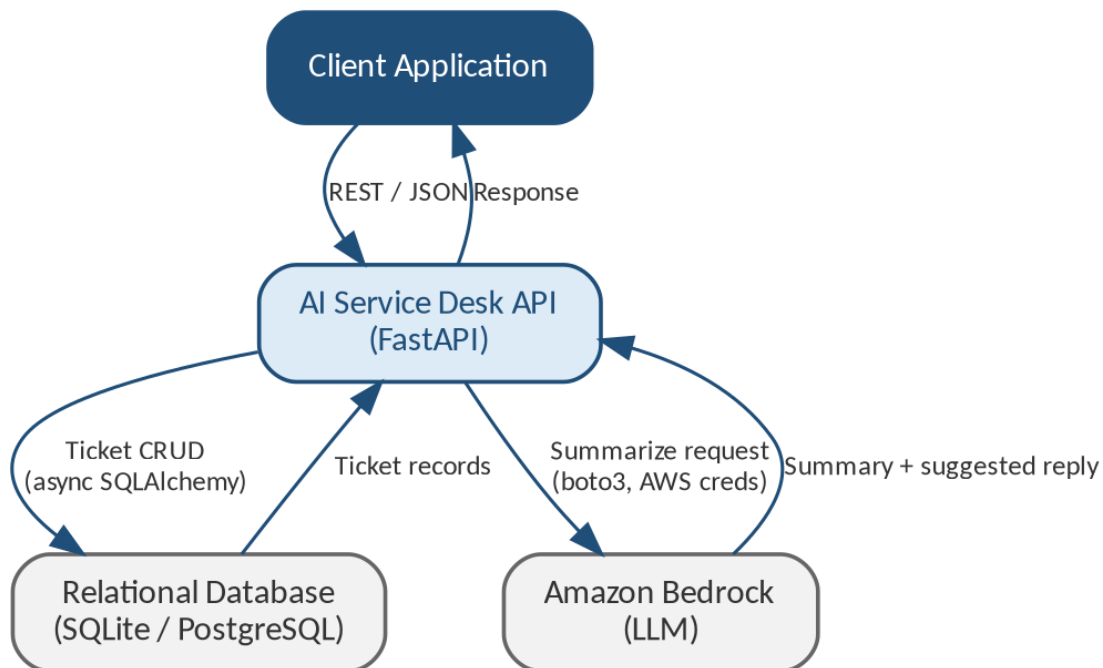
- Amazon Bedrock – hosted foundation model used to summarize tickets and draft suggested responses.
- Relational database – SQLite (aiosqlite) for local development/tests; PostgreSQL (asyncpg/psycopg) supported for other environments via DATABASE\_URL.

## 6.2 APIs Used

The service calls the Amazon Bedrock Runtime “Converse” API (boto3 bedrock-runtime client) to generate structured JSON containing a summary and suggested\_response for a given ticket description. The target model is configured via the BEDROCK\_MODEL\_ID setting, and the prompt is defined in a versioned template (TICKET\_SUMMARY\_V1).

## 6.3 Data Flow Diagram

Client → FastAPI (/ai/summarize) → BedrockService.summarize\_ticket() → prompt rendered from TICKET\_SUMMARY\_V1 → Amazon Bedrock Converse API → JSON response parsed and validated → SummarizeResponse (summary, suggested\_response) returned to the client. Ticket CRUD flows: Client → FastAPI (/tickets\*) → TicketService → TicketRepository → SQLAlchemy async engine → database.



## 6.4 Authentication/Authorization

Outbound calls to AWS Bedrock are authenticated using standard AWS credentials (access key, secret key, region) loaded from environment variables / .env via python-dotenv and passed explicitly to the boto3 client. There is no inbound authentication/authorization on the API's own endpoints at this time (see sections 2.3 and 4.2).

## 7. Test Strategy

### 7.1 Test Scenarios and Expected Results

| Test Scenario ID | Feature/Module    | Test Scenario                                                     | Expected Result                                                        |
|------------------|-------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| TS-01            | Ticket Management | Create a ticket with only required fields (minimum valid payload) | Ticket is created successfully with default values.                    |
| TS-02            | Ticket Management | Create a ticket with all valid fields                             | Ticket is created successfully.                                        |
| TS-03            | Ticket Management | Create multiple tickets                                           | All tickets are created successfully                                   |
| TS-04            | Ticket Management | Create a ticket with invalid or blank title                       | Validation error is returned and transaction is rolled back.           |
| TS-05            | Ticket Management | Validate title length and priority values                         | Input validation succeeds for valid values and rejects invalid values. |
| TS-06            | Ticket Management | Retrieve tickets when repository is empty                         | Empty list is returned successfully.                                   |
| TS-07            | Ticket Management | Retrieve an existing ticket using a valid ID                      | Correct ticket details are returned.                                   |
| TS-08            | Ticket Management | Retrieve filtered list of tickets                                 | Only tickets matching the filter criteria are returned.                |
| TS-09            | Ticket Management | Retrieve a ticket using an invalid ID                             | "Ticket not found" error is returned.                                  |

|       |                   |                                                    |                                                          |
|-------|-------------------|----------------------------------------------------|----------------------------------------------------------|
| TS-10 | Ticket Management | Verify computed ticket properties (is_resolved)    | Computed property returns the expected value.            |
| TS-11 | Ticket Management | Update a ticket without modifying any fields       | Existing ticket remains unchanged.                       |
| TS-12 | Ticket Management | Update a single ticket field                       | Requested field is updated successfully.                 |
| TS-13 | Ticket Management | Update multiple ticket fields                      | All specified fields are updated successfully.           |
| TS-14 | Ticket Management | Update invalid or closed ticket                    | Appropriate validation or business                       |
| TS-15 | Ticket Management | Validate allowed ticket status transitions         | Only valid status transitions are accepted.              |
| TS-16 | Ticket Management | Delete when repository contains no matching record | Delete operation returns appropriate failure response.   |
| TS-17 | Ticket Management | Delete an existing ticket                          | Ticket is deleted successfully.                          |
| TS-18 | Ticket Management | Delete multiple tickets                            | All selected tickets are deleted successfully.           |
| TS-19 | Ticket Management | Delete a non-existing ticket                       | Appropriate error is returned and rollback is performed. |
| TS-20 | Ticket Management | Delete ticket using valid UUID                     | Ticket is deleted successfully using UUID identifier.    |
| TS-21 | Ticket Management | Create ticket through REST API                     | HTTP 201 response with created ticket is returned.       |

|       |                   |                                            |                                                                       |
|-------|-------------------|--------------------------------------------|-----------------------------------------------------------------------|
| TS-22 | Ticket Management | Create ticket with different priorities    | Tickets are created successfully for low, medium and high priorities. |
| TS-23 | Ticket Management | Create ticket with invalid request payload | HTTP 422 validation error is returned.                                |
| TS-24 | Ticket Management | Retrieve ticket using REST API             | HTTP 200 response with ticket details is returned.                    |
| TS-25 | Ticket Management | Retrieve non-existing ticket               | HTTP 404 Not Found is returned.                                       |
| TS-26 | Ticket Management | Retrieve ticket list with filters          | Filtered ticket list is returned successfully.                        |
| TS-27 | Ticket Management | Validate query parameter combinations.     | Filter combinations return expected results.                          |
| TS-28 | Ticket Management | Update ticket through REST API             | Ticket is updated successfully.                                       |
| TS-29 | Ticket Management | Update non-existing ticket                 | HTTP 404 Not Found is returned.                                       |
| TS-30 | Ticket Management | Reopen a closed ticket                     | HTTP 400 Bad Request is returned according to business rule.          |
| TS-31 | Ticket Management | Update ticket with invalid request data    | HTTP 422 validation error is returned.                                |
| TS-32 | Ticket Management | Delete existing ticket                     | HTTP 200/204 response confirms                                        |
| TS-33 | Ticket Management | Delete non- existing ticket                | HTTP 404 Not Found is returned.                                       |
| TS-34 | Ticket Management | Delete ticket with invalid UUID            | HTTP 422 validation error is returned.                                |

|       |                   |                         |                                                                          |
|-------|-------------------|-------------------------|--------------------------------------------------------------------------|
| TS-35 | Ticket Management | Access /health endpoint | Application status is reported as healthy.                               |
| TS-36 | Ticket Management | Access /ready endpoint  | Application readiness status is returned based on database availability. |

### 7.3 Traceability to Requirements

| Requirement ID | Requirement Description               | Requirement Type | Mapped Test Scenario(s)                                       |
|----------------|---------------------------------------|------------------|---------------------------------------------------------------|
| FR-01          | Create a support ticket               | Functional       | TS-01, TS-02, TS-03, TS-04, TS-05, TS-21, TS-22, TS-23        |
| FR-02          | Retrieve all tickets                  | Functional       | TS-06, TS-08, TS-26, TS-27                                    |
| FR-03          | Retrieve a ticket by ID               | Functional       | TS-07, TS-09, TS-24, TS-25                                    |
| FR-04          | Update an existing ticket             | Functional       | TS-11, TS-12, TS-13, TS-14, TS-15, TS-28, TS-29, TS-30, TS-31 |
| FR-05          | Delete a ticket                       | Functional       | TS-16, TS-17, TS-18, TS-19, TS-20, TS-32, TS-33, TS-34        |
| FR-06          | Monitor application health            | Functional       | TS-35                                                         |
| FR-07          | Verify application readiness          | Functional       | TS-36                                                         |
| NFR-01         | Validate user input before processing | Non-Functional   | TS-04, TS-05, TS-23, TS-31, TS-34                             |

|        |                                                   |                |                                                               |
|--------|---------------------------------------------------|----------------|---------------------------------------------------------------|
| NFR-02 | Provide reliable error handling and API responses | Non-Functional | TS-09, TS-14, TS-19, TS-25, TS-29, TS-30, TS-33, TS-35, TS-36 |
| NFR-03 | Ensure CRUD operations maintain data integrity    | Non-Functional | TS-03, TS-08, TS-13, TS-18, TS-21, TS-26, TS-28, TS-32        |

## 8. Deployment & Configuration

### 8.1 Environments

- Local development – SQLite (aiosqlite) via DATABASE\_URL, run with uvicorn directly.
- Containerized – Docker image built from the provided Dockerfile (Python 3.12-slim base), running as a non-root user, exposing port 8000.
- Database-backed environments (e.g., staging/production) – PostgreSQL via asyncpg or psycopg, configured through DATABASE\_URL.

### 8.2 Setup Steps

Local (non-Docker) setup:

```
git clone <repository-url>
cd ai-service-task
uv sync          # or: pip install -r requirements.txt
cp .env.example .env # populate required variables (see 8.3)
alembic upgrade head # apply database migrations
uvicorn main:app --reload
```

Docker setup:

```
docker build -t ai-service-desk .
docker run -p 8000:8000 --env-file .env ai-service-desk
```

Testing:

```
pytest          # run unit + integration tests
python cprofile_runner.py # run performance profiling
locust -f locustfile.py  # run load tests
```



## 8.3 Configuration Files and Parameters

| Variable                                     | Purpose                                                           |
|----------------------------------------------|-------------------------------------------------------------------|
| APP_NAME                                     | Application display name                                          |
| API_VERSION                                  | API version identifier                                            |
| DEBUG                                        | Enables debug behavior                                            |
| DATABASE_URL                                 | SQLAlchemy async database connection string                       |
| SECRET_KEY                                   | Reserved for future token-based authentication                    |
| ACCESS_TOKEN_EXPIRE_MINUTES                  | Reserved for future token expiry configuration                    |
| AWS_ACCESS_KEY_ID /<br>AWS_SECRET_ACCESS_KEY | AWS credentials for the Bedrock client                            |
| AWS_REGION                                   | AWS region for the Bedrock client                                 |
| AWS_DEMO_MODE                                | Flag to toggle a demo/offline mode (used with FakeBedrockService) |
| DATABASE_READY                               | Flag indicating database readiness                                |
| BEDROCK_MODEL_ID                             | Identifier of the Bedrock foundation model to invoke              |

These values are loaded via Pydantic Settings (app/core/config.py) from a .env file at the project root; alembic.ini and alembic/env.py configure schema-migration behavior separately.

## 9. Appendix

### Glossary

| Term                 | Meaning in this project                                                                        |
|----------------------|------------------------------------------------------------------------------------------------|
| Ticket               | A support request tracked by the system, with a title, priority, status, and optional assignee |
| Summarization        | The AI-generated condensed description of a ticket, plus a suggested reply                     |
| Bedrock              | Amazon's managed service for accessing foundation (LLM) models                                 |
| Repository (pattern) | The data-access layer isolating SQLAlchemy queries from business logic                         |

## 1.POST Method

### Responses

**Curl**

```
curl -X 'POST' \
  'http://127.0.0.1:8000/tickets' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "assignment",
    "priority": "high"
  }'
```

**Request URL**

```
http://127.0.0.1:8000/tickets
```

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   "id": "e3b9ca24-e060-4b5a-8d52-79b662b6be76",   "title": "assignment",   "priority": "high",   "status": "open",   "created_at": "2026-07-24T04:57:45",   "is_resolved": false }</pre> <p><b>Response headers</b></p> <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 155 content-type: application/json date: Fri, 24 Jul 2026 04:57:44 GMT server: uvicorn vary: Origin x-response-time: 392.82ms</pre> |

**Responses**

## 2.GET Method

### Responses

**Curl**

```
curl -X 'GET' \
  'http://127.0.0.1:8000/tickets' \
  -H 'accept: application/json'
```

**Request URL**

```
http://127.0.0.1:8000/tickets
```

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p><b>Response body</b></p> <pre>{   {     "id": "e3b9ca24-e060-4b5a-8d52-79b662b6be76",     "title": "assignment",     "priority": "high",     "status": "open",     "created_at": "2026-07-24T04:57:45",     "is_resolved": false   } }</pre> <p><b>Response headers</b></p> <pre>content-length: 157 content-type: application/json date: Fri, 24 Jul 2026 04:58:25 GMT server: uvicorn x-response-time: 30.45ms</pre> |

**Responses**

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

### 3.GET by ticket\_id

GET
/tickets/{ticket\_id}
Get Ticket

Parameters
Cancel

| Name                                                                                                                      | Description                                                       |
|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>ticket_id</b> <span style="color: red;">*</span> <span style="color: red;">required</span><br>string(\$uuid)<br>(path) | <input type="text" value="e3b9ca24-e060-4b5a-8d52-79b662b6be76"/> |

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76
```

Server response

### 4.PATCH Method

Curl

```
curl -X 'PATCH' \
'http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "title": "assignment",
  "priority": "low",
  "status": "open",
  "assignee": "string"
}'
```

Request URL

```
http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76
```

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div> Response body <pre>{   "id": "e3b9ca24-e060-4b5a-8d52-79b662b6be76",   "title": "assignment",   "priority": "low",   "status": "open",   "created_at": "2026-07-24T04:57:45",   "is_resolved": false }</pre> Download </div> <div> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 154 content-type: application/json date: Fri, 24 Jul 2026 05:06:02 GMT server: uvicorn vary: Origin x-response-time: 218.39ms</pre> </div> |

Responses

## 5.DELETE Method

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/tickets/e3b9ca24-e060-4b5a-8d52-79b662b6be76
```

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><div><pre>{   "message": "Ticket deleted successfully" }</pre></div><div><div>Download</div></div></div> <div><div>Response headers</div><div><pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 41 content-type: application/json date: Fri, 24 Jul 2026 05:06:43 GMT server: uvicorn vary: Origin x-response-time: 50.06ms</pre></div></div> |

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

## 6.AI POST Method

|                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Curl                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| <pre>curl -X 'POST' \   http://127.0.0.1:8000/ai/summarize' \   -H 'accept: application/json' \   -H 'Content-type: application/json' \   -d '{     "ticket_description": "no tickets"   }'</pre> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| Request URL                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| <pre>http://127.0.0.1:8000/ai/summarize</pre>                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| Server response                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| Code                                                                                                                                                                                              | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |       |
| <hr/>                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| 200                                                                                                                                                                                               | <div><div>Response body</div><div><pre>{   "summary": "The support ticket was submitted with no content or details provided.",   "suggested_response": "Thank you for reaching out to our support team. It appears your message was submitted without any content. Could you please provide more details about your issue or question so we can assist you effectively?" }</pre></div><div><div>Download</div></div></div> <div><div>Response headers</div><div><pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 314 content-type: application/json date: Fri, 24 Jul 2026 05:15:30 GMT server: uvicorn vary: Origin x-response-time: 5336.06ms</pre></div></div> |       |
| Responses                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |       |
| Code                                                                                                                                                                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Links |

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/health' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/health
```

Server response

| Code | Details                                                                                                                                                                                                                 |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "status": "ok" }</pre> <p>Response headers</p> <pre>content-length: 15 content-type: application/json date: Fri, 24 Jul 2025 05:16:14 GMT server: uvicorn x-response-time: 111.97ms</pre> |

Responses

## 8.READY

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/ready' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/ready
```

Server response

| Code | Details                                                                                                                                                                                                                  |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "status": "ready" }</pre> <p>Response headers</p> <pre>content-length: 18 content-type: application/json date: Fri, 24 Jul 2025 05:17:03 GMT server: uvicorn x-response-time: 2.46ms</pre> |

Responses

| Code | Description                                                                                                                              | Links    |
|------|------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | <p>Successful Response</p> <p>Media type: <span>application/json</span></p> <p>Controls: Accept header</p> <p>Example Value   Schema</p> | No links |

## Code Snippets

### 1. Ticket Route

```
@router.post(
    "/tickets",
    response_model=TicketResponse,
)
async def create(
    ticket: TicketCreate,
    db: AsyncSession = Depends(get_db),
):
    return await ticket_service.create_ticket(
        db,
        ticket,
    )
```

### 2. Ticket Schema

```
class TicketResponse(BaseModel):
    id: UUID
    title: str
    priority: Literal["low", "medium", "high"]
    status: Literal["open", "in_progress", "resolved", "closed"]
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)

    @computed_field
    @property
    def is_resolved(self) -> bool:
        return self.status == "resolved"
```

### 3. Ticket Service

```
async def delete_ticket(
    self,
    db,
    ticket_id,
):
    ticket = await ticket_repository.get_ticket(
        db,
        ticket_id,
    )

    if ticket is None:
        raise TicketNotFoundError(str(ticket_id))

    await ticket_repository.delete_ticket(
        db,
        ticket_id,
    )

    return True
```

## 4.AI Route

```
@router.post("/summarize", response_model=SummarizeResponse)
def summarize_ticket(payload: SummarizeRequest):
    try:
        return service.summarize_ticket(payload.ticket_description)
    except BedrockServiceError as exc:
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail="The AI service is temporarily unavailable",
        ) from exc
```

## References

- Project source repository (ai-service-task)
- FastAPI – <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 async ORM – <https://docs.sqlalchemy.org>
- Alembic – <https://alembic.sqlalchemy.org>
- Amazon Bedrock Runtime (boto3) – <https://docs.aws.amazon.com/bedrock/>
- Locust load testing – <https://locust.io>